



SNA-GraphSAGE

Social Network Analysis with Graph Neural Networks

Part 2 of 2 — **Codebase & File Guide**

What each module does, and how data flows from dataset to results.

Prepared 14 July 2026 · Author: Bhanu Prakash

o • The folder map

A stage-based experiment runner (`main.py`) driving a small library of models, experiments and utilities. Numeric results merge into JSON; figures render to `figures/`.

```
SNA-GraphSAGE/
├─ main.py                # stage-based experiment runner (argparse)
├─ config.py              # all hyperparameters + paths
├─ requirements.txt       # torch, torch-geometric, sklearn, networkx...
├─ Enhanced_SNA_GNN_Paper.pdf # the project's own write-up
├─ MPResearch.pdf         # the reference paper (Xiao et al. 2019)
├─ models/
│   ├── gnn_zoo.py         # unified GNN: sage/gcn/gat/gin
│   ├── link_predictor.py  # SAGE encoder + dot-product decoder
│   ├── deepwalk_model.py  # random walks + Word2Vec baseline
│   ├── logistic_regression.py # content-only LR baseline
│   └─ graphsage_model.py  # legacy standalone SAGE (superseded)
├─ experiments/
│   ├── model_comparison.py # architecture + aggregator studies
│   ├── feature_ablation.py # raw vs SNA vs raw+SNA
│   ├── layer_experiments.py # depth / oversmoothing
│   ├── sampling_experiments.py # degree-adaptive vs fixed sampling
│   └─ community_detection.py # Louvain vs KMeans variants
├─ utils/ data_loader, structural_features, train_utils, metrics, visualization
└─ data/ results/ figures/ paper/
```

Data flow: `load_planetoid` → (optional) `augment_with_structural_features` (concat 8 SNA centralities) → `train_node_classifier` (full-batch Adam, best-val checkpoint, early stop) wrapped in `run_multi_seed` → `full_metrics` (accuracy, micro/macro-F1, confusion) → `visualization` (PCA/t-SNE, curves, bars) → merged into `results/results_<dataset>.json`.

1 • Core & models

ENTRY POINT

main.py

Argparse stage runner: nine `stage_*` functions (baselines, models, aggregators, features, depth, labelrate, sampling, linkpred, community). Orchestrates load→run→save and writes merged JSON. `--dataset`, `--stages`, `--quick` flags.

CONFIGURATION

config.py

All hyperparameters + paths: seed list [42,43,44], device, dataset, GNN hyperparams (hidden 128, dropout 0.5, lr 0.01, wd 5e-4, epochs 300, patience 30, GAT heads 8), DeepWalk params, betweenness pivot count, adaptive-sampling knobs (c=5, w=0.5, cap=25), link-pred dims, t-SNE perplexity, output dirs.

MODELS/	PURPOSE
<code>gnn_zoo.py</code>	The unified <code>GNN</code> class (arch = sage/gcn/gat/gin), configurable layers/dropout/aggregator/heads; <code>get_embeddings()</code> returns the penultimate layer. Used by every experiment.
<code>link_predictor.py</code>	<code>LinkPredictor</code> (SAGE encoder + dot-product decode) and <code>run_link_prediction</code> (RandomLinkSplit, per-epoch negative sampling, AUC/AP).
<code>deepwalk_model.py</code>	Builds a NetworkX graph, generates random walks, trains gensim Word2Vec → structure-only baseline embeddings.
<code>logistic_regression.py</code>	scikit-learn LogisticRegression on masked features — content-only baseline.
<code>graphsage_model.py</code>	Legacy standalone GraphSAGE + trainer, superseded by <code>gnn_zoo.py</code> (kept for reference).

2 • Experiments & utils

EXPERIMENTS/	PURPOSE
<code>model_comparison.py</code>	SAGE/GCN/GAT/GIN multi-seed comparison + aggregator study (mean/max/sum/attention); saves bar charts.
<code>feature_ablation.py</code>	Raw vs SNA-only vs raw+SNA feature variants.
<code>layer_experiments.py</code>	Depth 1–4 (oversmoothing) study.
<code>sampling_experiments.py</code>	Hand-rolled layered-BFS neighbour sampler; <code>adaptive_sample_size</code> implements the degree rule; fixed vs adaptive comparison.
<code>community_detection.py</code>	Louvain vs KMeans-raw vs KMeans-on-embeddings, scored by NMI/ARI, with optional t-SNE plots.

UTILS/	PURPOSE
<code>data_loader.py</code>	<code>load_planetoid</code> / <code>load_cora</code> , <code>set_seeds</code> , <code>augment_with_structural_features</code> .
<code>structural_features.py</code>	Computes 8 z-scored NetworkX centralities; caches to <code>.npy</code> .
<code>train_utils.py</code>	<code>train_node_classifier</code> (full-batch, early stop, best-val restore) + <code>run_multi_seed</code> .
<code>metrics.py</code>	<code>evaluate_logits</code> / <code>full_metrics</code> (accuracy, micro/macro-F1, confusion, report).
<code>visualization.py</code>	PCA/t-SNE scatter, training curves, confusion matrix, comparison bars → <code>figures/</code> .

3 • Commands

COMMAND	WHAT IT DOES
<code>pip install -r requirements.txt</code>	Install deps (PyG, sklearn, networkx, gensim).
<code>python main.py</code>	Run all stages on Cora.
<code>python main.py --stages models,community</code>	Run only selected stages.
<code>python main.py --dataset CiteSeer</code>	Switch dataset (Cora / CiteSeer / PubMed).
<code>python main.py --quick</code>	Single seed, fewer epochs, skip t-SNE (fast smoke test).

Outputs: figures → `figures/*.png` ; numeric metrics → `results/results_<dataset>.json` (partial runs merge, not clobber); SNA-feature cache → `data/structural_<dataset>.npy` . Datasets auto-download via PyG's Planetoid loader.